(Page 403 onwards)

Reduce side Joins and Grouping

Lets say an events stream is there. Clicks event.

User Activity Events.

userid	eventid
user123	video23
user243	video05

clickstream

Users Table

userid	age	gender
user123	26	M
user243	25	F

Users table

Lets say we have a clickstream, userid and videoid on which video they clicked and we want to find age of that userid.

(P.T.O)

Let's say analytics want to know ~~which age gp~~ which video is common b/w which age group. You have to find this.

So you will basically join the two tables by userid.

userid	videoId	age

⇒ then you have to group by age and then group by videoId and count and have to do an order by DESC and then do a Limit 1.

1st Approach.

As dataset is brought into system, and event goes to each map task. For each record we can query the database for the age → ✗ very bad approach.

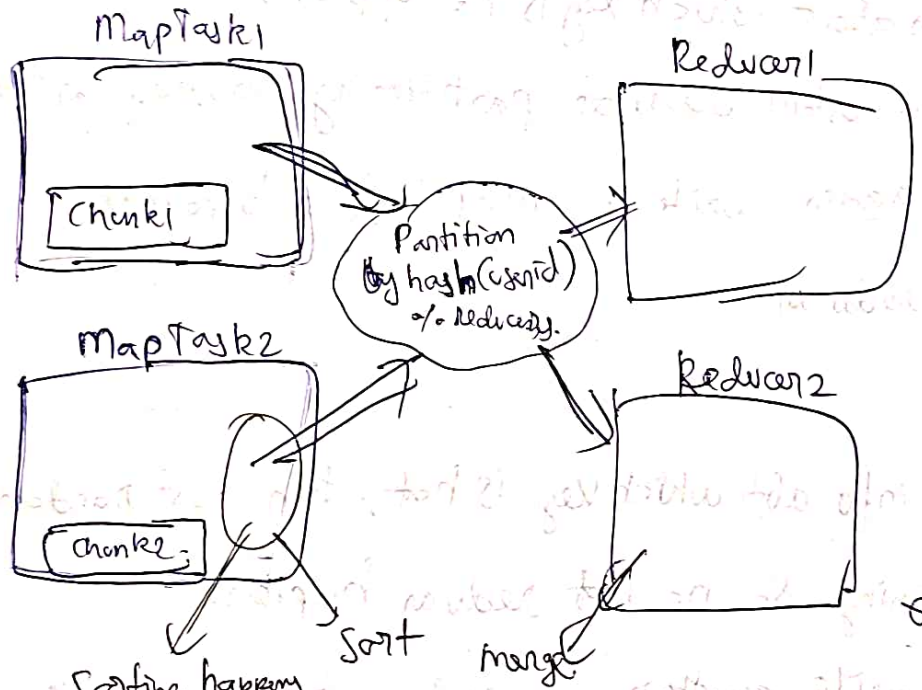
1) Network Call for each userid
2) Random Access if no indexing
3) For each record putting huge load on DB.

Few ways to Solve this.

Sort-Merge Joins

In this approach, we load both the user and activity table into HDFS, ~~and~~ ~~store them partitioned by user~~

So now what happens.



So basically events and user data go to one reducer.

So each Reducer has its ~~own partition~~ own set of user records.

So reducer can merge records for 1 user together.

↓

Hence (sort-merge join)

(P.T.O)

Sort-merge join is a Reducer side join.

Problem in one case of hot-key.

Any userid becomes hot and that reducer is becoming bombarded with events.

~~Approach to solve it~~

Case 1

→ If you have details about which key is hot upfront, then introduced a custom reducer partitioning strategy for that key and later again write a map reduce to collate result for that reducer.

2nd Case

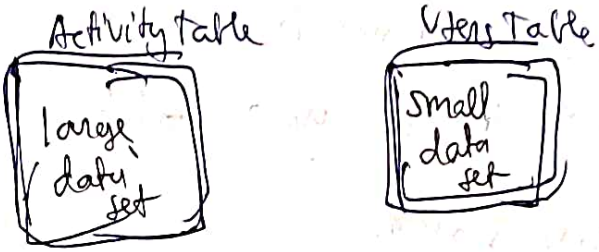
→ If you don't have info abt which key is hot, then just randomise the ~~process~~ partitioning so no hot reducer happens.
Then similarly write another mapreduce task to gather events, this time data will be grouped hence will be smaller.

Map Side Merges.

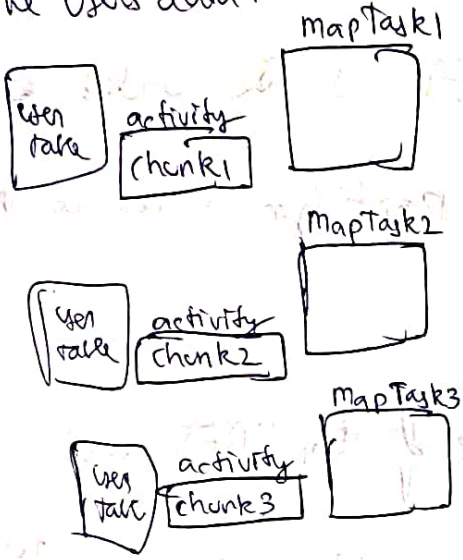
This was Map Side merges, how abt we do the joins in the mapper itself, not load on reducer. and no need for Reducers to make network call to fetch file.

3 Strategies for Map-Side Joins.

1) Broadcast hash joins (only works when 1 small dataset)



→ In this approach, we assume the users data is small,

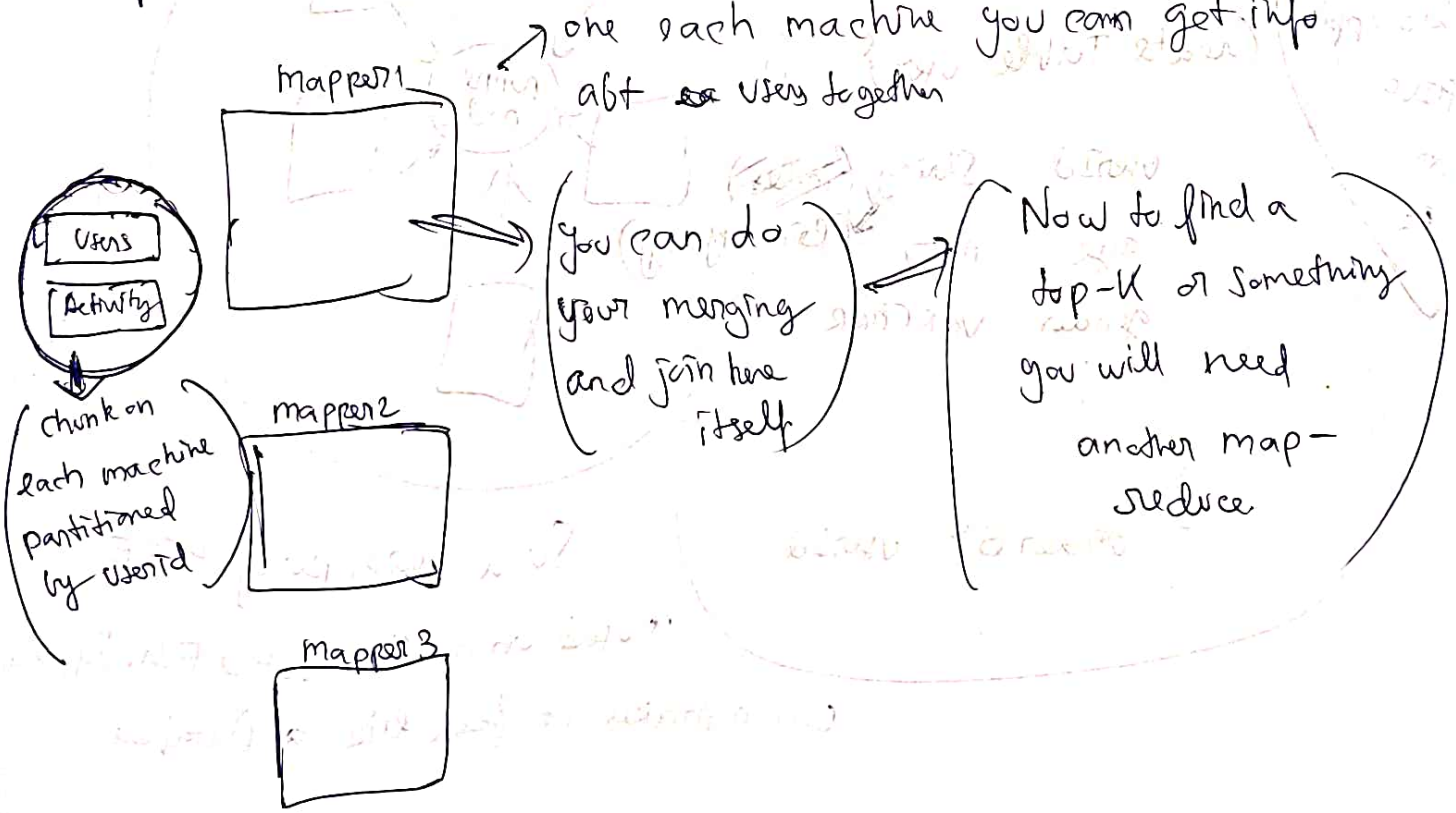


So in this, the users table will be in-memory in each of the mapper instance.

And joins can happen easily

2) Partitioned Hash Joins. →

This works by partitioning and keeping in the mapper the partitioned data

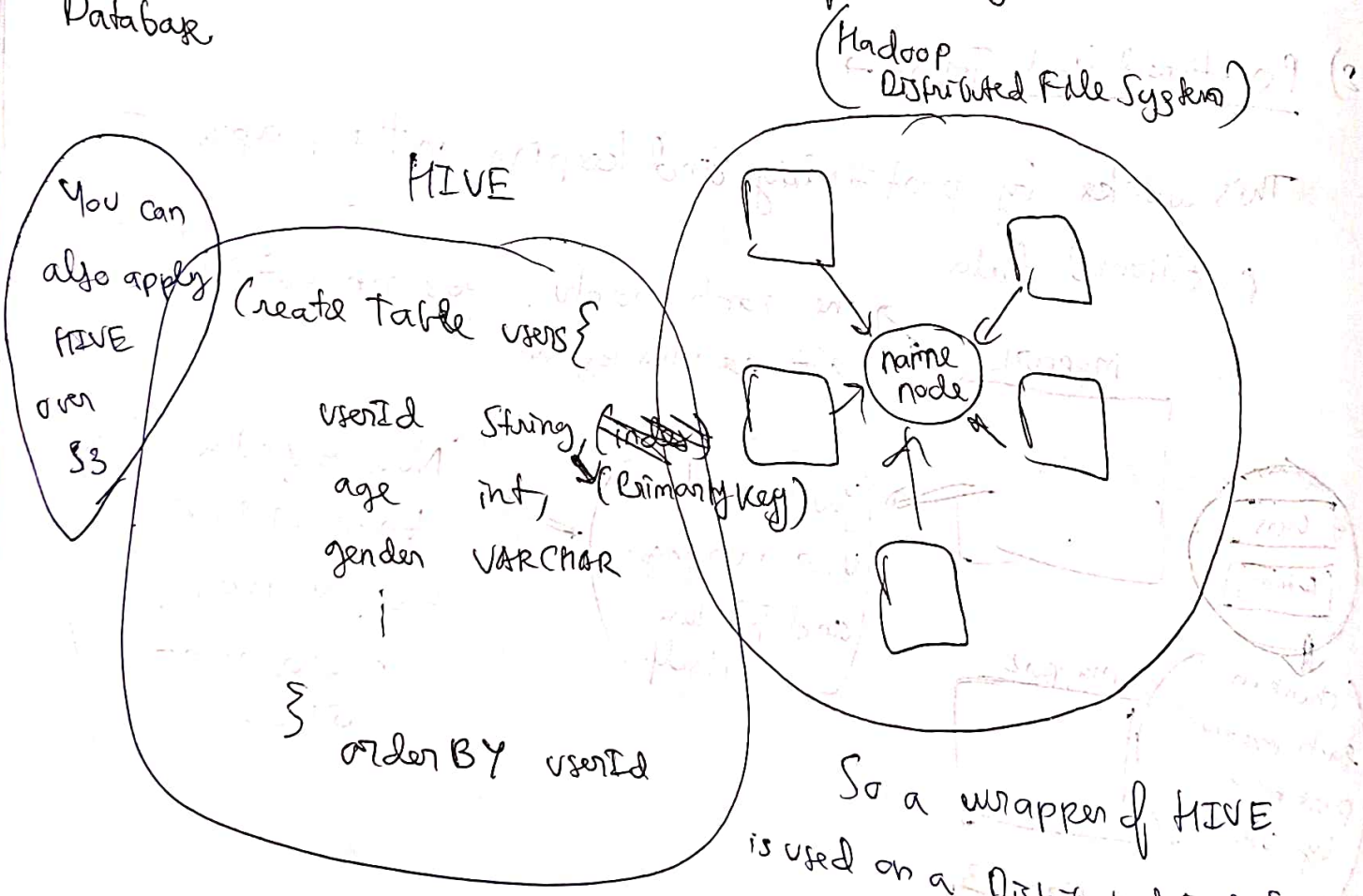


(Map-side merges)

Same as partitioned has,
But in this one your table is also sorted, I mean the chunk is also sorted.

I asked Chatgpt does it mean the table is Loaded on the machine and then whichever partition is there on the machine, that is sorted?

Then chatgpt cleared the architecture of HDFS and how HIVE provides an illusion of making it seem like a Database



So a wrapper of HIVE is used on a Distributed File System, which makes it feel like a Database

Batch Processing

Map Reduces typically works on huge volumes of data, dividing work into small chunks (via mappers) then aggregating (using reducers).

However the output needs to be stored in such a format so that it is queriable to display.

Use cases.

1). Building Search Indexes → After map Reduce google can use to build its entire search index.
So it reuploads the entire index and maybe a job runs daily

2). Key-value Stores → After Batch processing as we saw [url → count] & making it seem like a (key, value) Datastore

3). Avoid writing to Database → First of all the Data itself is Super huge and secondly external Database won't be able to handle such high write throughput, so as map-Reduce does writing into local disk of Reducer.

5) UNIX Philosophy → Map Reduce follow the UNIX philosophy of storing outputs in files and also they are immutable so, thats easier to maintain

Few Limitations of Map Reduces

* Map Reduce saves intermediate state in File (materialization) while this allows fault tolerance but has an overhead of writing to disk as well.

* A job's completion is completely dependent on the reducer which is taking maximum completion time.

* Spark, Flink other DataFlow Engines thrive over Map Reduce today
  - No intermediate materialisation
  - Flink does periodic writes to disk for checkpointing
  - Spark Does the whole job (task) in 1 job
    whereas otherwise you would need multiple Map-Reduce